Name : A .Purusothaman

Reg.No:192421405

Subject: Ethical Hacking

Sub.Code :ITA1407

CO4 -ASS1-ETHICAL HACKING

Question 1 — Web Server Reconnaissance and Vulnerability Analysis

Theory

Web-server reconnaissance is the process of collecting information about a server before vulnerability assessment. The objective is to identify the server's IP address, open ports, running services, software versions, web technologies, directories, files, and potentially exposed information.

Common tools include:

- **WHOIS** — obtains domain-registration information.
- **Ping** — checks basic network reachability.
- **Traceroute/Tracert** — shows the network path to a host.
- **IPConfig/IP** — displays local network configuration.
- **Netstat/ss** — displays network connections and listening ports.
- **Nmap** — discovers open ports and identifies services/versions.
- **WhatWeb** — identifies web technologies and frameworks.
- **Curl/Wget** — examines HTTP responses and headers.

- **Nikto** — checks for common web-server vulnerabilities and misconfigurations.
- **Gobuster/Dirb** — discovers directories and files.

Commands

Identify your own VM network configuration

ip addr

Windows equivalent

ipconfig

Test connectivity

ping <LAB_IP>

Linux

traceroute <LAB_IP>

Windows

tracert <LAB_IP>

Linux

ss -tulpen

Alternative

netstat -tulpen

WHOIS

whois <LAB_DOMAIN>

For an isolated VM, WHOIS may not return useful information if the target is only a private IP.

Nmap

nmap <LAB_IP>

Service/version detection:

nmap -sV <LAB_IP>

Scan all TCP ports:

nmap -p- <LAB_IP>

Combined:

nmap -sV -p- <LAB_IP>

WhatWeb

whatweb http://<LAB_IP>

HTTP headers

curl -I http://<LAB_IP>

More detailed response:

curl -i http://<LAB_IP>

Nikto

nikto -h http://<LAB_IP>

Gobuster

gobuster dir \

-u http://<LAB_IP> \

-w /usr/share/wordlists/dirb/common.txt

If your Kali installation uses another wordlist:

ls /usr/share/wordlists/

Observations to record

Record things such as:

- Open ports: 80, 443, 22, etc.
- Web-server software and version.
- Application/framework identified by WhatWeb.
- HTTP response headers.

- Missing security headers.
- Interesting directories discovered by Gobuster.
- Files or administrative interfaces discovered by Nikto/Gobuster.
- Whether software appears outdated.

Do not assume a tool's warning is automatically a confirmed vulnerability. Validate findings against the actual application.

Question 2 — Web Application Vulnerability Assessment

Theory

Web-application vulnerability assessment examines how an application handles user input, authentication, sessions, requests, and authorization.

Important vulnerability classes include:

SQL Injection

Occurs when untrusted input is incorporated into SQL queries without appropriate parameterization.

Potential impact:

- Unauthorized database access
- Data disclosure
- Data modification
- Authentication bypass

Remediation: use parameterized queries/prepared statements and avoid constructing SQL queries through string concatenation.

Cross-Site Scripting (XSS)

Occurs when attacker-controlled content is interpreted as executable JavaScript in a victim's browser.

Types include:

- Reflected XSS

- Stored XSS
- DOM-based XSS

Remediation: contextual output encoding, input validation, appropriate CSP, and safe framework APIs.

Authentication weaknesses

Examples:

- Weak passwords
- Missing brute-force protection
- Improper login validation
- Authentication bypass

Session-management weaknesses

Examples:

- Predictable session IDs
- Missing Secure/HttpOnly attributes
- Sessions that do not expire appropriately
- Session fixation

Directory traversal

A vulnerable application may allow user input to influence filesystem paths.

Potential impact is unauthorized access to files.

Tools

Use:

- Burp Suite
- OWASP ZAP
- Browser Developer Tools
- Nmap
- Curl

- Nikto

Basic ZAP workflow

1. Start OWASP ZAP.
2. Configure the browser to use ZAP's proxy.
3. Browse the authorized laboratory application.
4. Allow ZAP to capture requests.
5. Examine:
    - URL parameters
    - POST parameters
    - Cookies
    - Headers
    - Response codes
    - Session identifiers
6. Run the automated scanner only against the authorized lab application.
7. Manually validate selected findings.

Curl example

curl -i http://<LAB_IP>/

Testing a known lab endpoint:

curl -i "http://<LAB_IP>/search?q=test"

POST request:

curl -i -X POST \

-d "username=test&password=test" \

http://<LAB_IP>/login

For SQLi/XSS testing, use **only the faculty-provided vulnerable application and test parameters**. Record the request and response rather than attempting destructive database operations.

Question 3 — Security Headers and HTTPS

Theory

Security headers instruct browsers how web content should be handled.

Content-Security-Policy

CSP restricts the sources from which scripts and other resources can be loaded.

Example:

Content-Security-Policy: default-src 'self'

HSTS

HTTP Strict Transport Security instructs browsers to use HTTPS.

Example:

Strict-Transport-Security: max-age=31536000; includeSubDomains

X-Frame-Options

Helps prevent clickjacking.

Example:

X-Frame-Options: DENY

X-Content-Type-Options

Prevents MIME-type sniffing.

Example:

X-Content-Type-Options: nosniff

Commands

Check HTTP:

curl -I http://<LAB_IP>

Check HTTPS:

curl -I https://<LAB_IP>

Check redirect:

curl -I http://<LAB_IP>

Look for:

HTTP/1.1 301

Location: https://...

Nmap TLS analysis

nmap -p 443 --script ssl-enum-ciphers <LAB_IP>

Certificate:

nmap -p 443 --script ssl-cert <LAB_IP>

Both:

nmap -p 443 --script "ssl-enum-ciphers,ssl-cert" <LAB_IP>

Things to document

Test	**Observation**
HTTP → HTTPS	Redirect / No redirect
CSP	Present / Missing
HSTS	Present / Missing
X-Frame-Options	Present / Missing
X-Content-Type-Options	Present / Missing
TLS 1.0/1.1	Enabled / Disabled
TLS 1.2/1.3	Enabled / Disabled
Weak ciphers	Found / Not found
Certificate	Valid / Problem identified

Question 4 — Broken Access Control

Theory

Access control determines **what an authenticated user is allowed to do**.

Authentication answers:

"Who are you?"

Authorization answers:

"What are you allowed to access?"

For example, a student may successfully authenticate but should not be able to access another student's record or an administrator-only page.

Testing methodology

Use the two **faculty-provided test accounts**.

For example:

Student A

Student B

Administrator

Log in as Student A and capture a request such as:

GET /student/profile?id=1001

Then determine whether the application exposes an object identifier.

Compare the server's behavior when using another **faculty-provided test identifier**.

Record:

- Request URL
- HTTP method
- Parameters
- Cookies/session
- HTTP status

- Response size/content
- Whether access was permitted or denied

For an administrator-only endpoint, test whether the normal student account receives an appropriate denial.

ZAP procedure

1. Log in using Student A.
2. Capture the request in ZAP.
3. Identify the object identifier.
4. Repeat the request using the faculty-provided test identifier.
5. Compare responses.
6. Log out.
7. Log in as Student B.
8. Repeat the authorized test.
9. Test administrator-only resources with the normal student account.
10. Document the result.

Do **not** enumerate arbitrary identifiers or attempt to access real users' records. Use only identifiers supplied for the laboratory exercise.

Severity

A broken access-control vulnerability can be **High or Critical** when it permits unauthorized access to sensitive records or administrative functionality.

Question 5 — Automated Scanning and Manual Validation

Theory

Automated scanners rapidly identify potential vulnerabilities, but scanner results are not necessarily confirmed vulnerabilities.

Therefore:

Automated detection → Manual validation → Classification → Remediation

Two tools can produce different results because they use different detection methods and databases.

Nikto

nikto -h http://<LAB_IP>

OWASP ZAP

Start ZAP:

zaproxy

Then:

1. Configure the target.
2. Browse/spider the authorized application.
3. Run the passive scan.
4. Run the active scan against the laboratory target.
5. Review alerts.
6. Export the report.

Alternative: Wapiti

wapiti -u http://<LAB_IP>/

Depending on the installed version, additional options can be viewed with:

wapiti --help

Example Finding Classification

Use this format in your report:

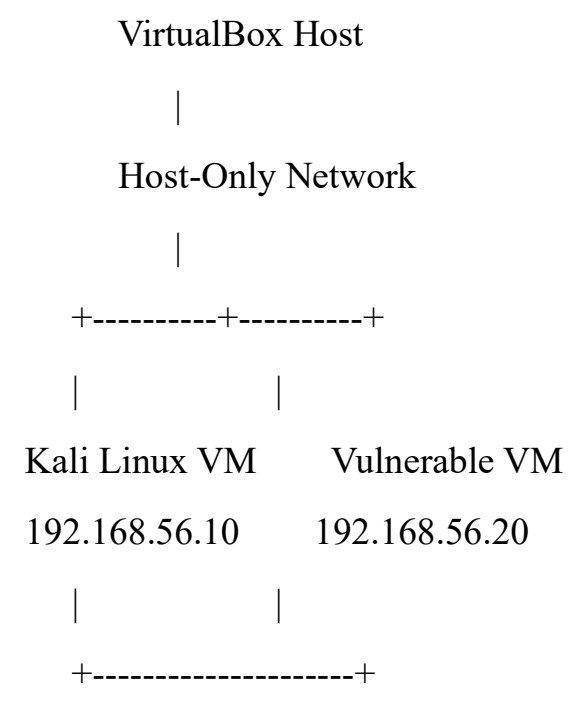
Finding	Tool	Evidence	Severity	Impact	Status
Missing CSP	Curl/ZAP	Header absent	Medium	Reduced XSS defense	Confirmed
Directory listing	Nikto	Directory exposed	Medium	Information disclosure	Confirmed

Finding	Tool	Evidence	Severity	Impact	Status
Outdated server	Nikto/Nmap	Version identified	Medium/High	Potential known vulnerabilities	Requires validation
XSS	ZAP/manual	Input reflected unsafely	High	Script execution in victim browser	Confirmed
Broken access control	ZAP	Unauthorized record returned	High	Unauthorized data disclosure	Confirmed

Severity should be based on **actual evidence, exploitability, and impact**, not simply the scanner's label.

Recommended VirtualBox Lab Layout

A safe configuration is:



Use **Host-Only Adapter** or another faculty-approved isolated network so the vulnerable machine is not exposed to the public Internet.

Check the target's address from the vulnerable VM:

ip addr

Then from Kali:

ping 192.168.56.20

and:

nmap -sV 192.168.56.20

Replace 192.168.56.20 with the actual IP assigned to your laboratory vulnerable VM.